BTS SIO — Option SLAM

# DOCUMENTATION TECHNIQUE

Projet Professionnel Encadré

## Application de réservation de places de parking

| | |
|---|---|
| **Technologie** | Laravel (PHP) — Framework MVC |
| **Type de projet** | Application web de réservation |
| **Contexte** | PPE BTS SIO — Portfolio |
| **Nature** | Projet de groupe |
| **Rédaction du document** | Valentin Semanchuk |
| **Dépôt GitHub** | github.com/Rivalzina75/projet_parking |

# Sommaire

1. Présentation du projet
2. Cahier des charges
3. Architecture technique
4. Modèle de données
5. Fonctionnalités développées
6. Structure du projet Laravel
7. Extraits de code commentés
8. Guide d'installation
9. Tests et recette
10. Bilan et compétences BTS SIO

# 1. Présentation du projet

## 1.1 Contexte

Dans le cadre du BTS SIO option SLAM, ce projet professionnel encadré (PPE) consiste à développer une application web de gestion et réservation de places de parking. Ce type de projet correspond au contexte réel d'une organisation (par exemple la Maison des Ligues de Lorraine) souhaitant informatiser la gestion de son parking.

L'application est développée sous le framework PHP Laravel, qui impose une architecture MVC rigoureuse et des bonnes pratiques de développement.

## 1.2 Objectifs

- Permettre aux utilisateurs de consulter les places disponibles et d'effectuer une demande de réservation.
- Permettre à l'administrateur d'attribuer des places aux utilisateurs, de gérer les réservations et les utilisateurs.
- Respecter les principes MVC imposés par Laravel.
- Sécuriser les accès via un système d'authentification.

## 1.3 Périmètre fonctionnel

**Utilisateurs cibles**
Deux types d'utilisateurs sont distingués : l'utilisateur standard (employé, résident) qui réserve une place, et l'administrateur qui gère l'ensemble du système.

# 2. Cahier des charges

## 2.1 Expression des besoins

Le commanditaire souhaite une application accessible depuis un navigateur web permettant :

### Côté utilisateur

- Création de compte et connexion sécurisée.
- Consultation de la liste des places disponibles.
- Soumission d'une demande de réservation.
- Consultation de l'historique de ses réservations.
- Annulation d'une réservation en attente.

### Côté administrateur

- Tableau de bord avec vue d'ensemble des réservations.
- Attribution d'une place à un utilisateur ayant fait une demande.
- Gestion des places (ajout, suppression, modification du statut).
- Gestion des utilisateurs (liste, désactivation).
- Refus ou validation d'une demande de réservation.

## 2.2 Contraintes techniques

| Contrainte | Détail |
|---|---|
| Langage back-end | PHP 7+ via le framework Laravel |
| Architecture | MVC (Modèle - Vue - Contrôleur) |
| Base de données | MySQL / MariaDB avec migrations Laravel |
| Front-end | HTML5, CSS3, Blade (moteur de templates Laravel) |
| Authentification | Système intégré Laravel Auth |
| Versioning | Git / GitHub |
| Environnement | Serveur local (XAMPP / Laragon) ou Ubuntu |

# 3. Architecture technique

## 3.1 Pattern MVC avec Laravel

Laravel impose le patron de conception MVC (Modèle-Vue-Contrôleur), ce qui garantit une séparation claire des responsabilités au sein de l'application.

| Couche | Rôle | Exemples dans le projet |
|---|---|---|
| Modèle (M) | Représente les données et les règles métier. Communique avec la BDD via Eloquent ORM. | User, Place, Reservation |
| Vue (V) | Gère l'affichage HTML à l'aide des templates Blade. | dashboard.blade.php, reservations/index.blade.php |
| Contrôleur (C) | Reçoit les requêtes HTTP, interagit avec les modèles et retourne les vues. | ReservationController, AdminController |

## 3.2 Flux d'une requête HTTP

1. L'utilisateur effectue une action (clic, soumission de formulaire).
2. Le routeur Laravel (routes/web.php) identifie le contrôleur concerné.
3. Le contrôleur appelle le ou les modèles nécessaires (via Eloquent).
4. Le modèle interroge la base de données MySQL et retourne les données.
5. Le contrôleur transmet les données à la vue Blade.
6. La vue génère le HTML renvoyé au navigateur de l'utilisateur.

## 3.3 Technologies et outils utilisés

| Outil / Technologie | Version | Usage |
|---|---|---|
| Laravel | 5.5+ | Framework PHP principal |
| PHP | 7.x | Langage back-end |
| MySQL | 5.7+ | Base de données relationnelle |
| Blade | — | Moteur de templates (vues) |
| Eloquent ORM | — | Abstraction de la base de données |
| Composer | — | Gestionnaire de dépendances PHP |
| Git / GitHub | — | Versioning et collaboration |
| Bootstrap | 3/4 | Framework CSS pour le front-end |

# 4. Modèle de données

## 4.1 Entités principales

La base de données comporte les tables suivantes, créées via les migrations Laravel :

### Table : users

| Champ | Type | Description |
|---|---|---|
| id | INT (PK) | Identifiant unique auto-incrémenté |
| name | VARCHAR(255) | Nom complet de l'utilisateur |
| email | VARCHAR(255) | Adresse e-mail (unique) |
| password | VARCHAR(255) | Mot de passe hashé (bcrypt) |
| role | ENUM('user','admin') | Rôle de l'utilisateur dans l'application |
| created_at / updated_at | TIMESTAMP | Horodatages automatiques Laravel |

### Table : places

| Champ | Type | Description |
|---|---|---|
| id | INT (PK) | Identifiant unique de la place |
| numero | VARCHAR(10) | Numéro ou libellé de la place (ex: A-12) |
| statut | ENUM('libre','occupée','réservée') | État courant de la place |
| created_at / updated_at | TIMESTAMP | Horodatages automatiques |

### Table : reservations

| Champ | Type | Description |
|---|---|---|
| id | INT (PK) | Identifiant unique de la réservation |
| user_id | INT (FK) | Référence vers l'utilisateur (users.id) |
| place_id | INT (FK) | Référence vers la place (places.id) |
| statut | ENUM('en attente','validée','refusée','annulée') | État de la demande |
| date_debut | DATE | Date de début de la réservation souhaitée |
| date_fin | DATE | Date de fin de la réservation souhaitée |

| Champ | Type | Description |
|---|---|---|
| created_at / updated_at | TIMESTAMP | Horodatages automatiques |

## 4.2 Relations entre entités

- Un utilisateur peut avoir plusieurs réservations (relation 1-N entre users et reservations).
- Une place peut être liée à plusieurs réservations dans le temps (relation 1-N entre places et reservations).
- Une réservation est liée à exactement un utilisateur et une place.

# 5. Fonctionnalités développées

## 5.1 Authentification

L'application utilise le système d'authentification intégré de Laravel (php artisan make:auth). Il gère :

- L'inscription d'un nouvel utilisateur avec validation des champs (email unique, confirmation du mot de passe).
- La connexion sécurisée avec protection CSRF.
- La déconnexion.
- La protection des routes via les middlewares auth et admin personnalisé.

## 5.2 Espace utilisateur

- Affichage de la liste des places disponibles.
- Formulaire de demande de réservation (sélection de la place, des dates).
- Tableau de bord personnel listant les demandes avec leur statut (en attente, validée, refusée).
- Possibilité d'annuler une demande en attente.

## 5.3 Espace administrateur

- Tableau de bord avec compteurs : nombre de places libres, demandes en attente, réservations actives.
- Liste de toutes les demandes de réservation avec possibilité de valider ou refuser chaque demande.
- Lors de la validation : attribution automatique d'une place disponible à l'utilisateur, mise à jour du statut de la place.
- Gestion des places : ajout d'une nouvelle place, modification du statut, suppression.
- Gestion des utilisateurs : visualisation de la liste, désactivation d'un compte.

## 5.4 Sécurité et validations

- Protection CSRF sur tous les formulaires (token @csrf de Blade).
- Validation des données côté serveur via les Form Requests Laravel.
- Middleware de vérification du rôle pour l'accès à l'espace admin.
- Hashage des mots de passe avec bcrypt.

# 6. Structure du projet Laravel

## 6.1 Arborescence des fichiers

```
projet_parking/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/                  # Contrôleurs d'authentification
│   │   │   ├── ReservationController.php  # Gestion des réservations
│   │   │   ├── PlaceController.php    # Gestion des places
│   │   │   └── AdminController.php    # Tableau de bord admin
│   │   └── Middleware/
│   │       └── IsAdmin.php            # Middleware de rôle
│   └── Models/ (ou directement dans app/)
│       ├── User.php
│       ├── Place.php
│       └── Reservation.php
├── database/
│   └── migrations/                    # Définition des tables
├── resources/
│   └── views/
│       ├── layouts/app.blade.php      # Layout principal
│       ├── auth/                      # Vues connexion/inscription
│       ├── reservations/              # Vues utilisateur
│       └── admin/                     # Vues administration
├── routes/
│   └── web.php                        # Définition des routes
└── .env                               # Configuration (BDD, app key...)
```

## 6.2 Routage (routes/web.php)

Les routes sont organisées en deux groupes principaux :

```php
// Routes publiques
Route::get('/', function () { return view('welcome'); });
Auth::routes();

// Routes authentifiées (utilisateur)
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', 'ReservationController@index');
    Route::post('/reservation', 'ReservationController@store');
    Route::delete('/reservation/{id}', 'ReservationController@destroy');
});

// Routes admin
Route::middleware(['auth', 'isAdmin'])->prefix('admin')->group(function () {
    Route::get('/dashboard', 'AdminController@index');
    Route::post('/reservation/{id}/valider', 'AdminController@valider');
    Route::post('/reservation/{id}/refuser', 'AdminController@refuser');
    Route::resource('places', 'PlaceController');
});
```

# 7. Extraits de code commentés

## 7.1 Modèle Reservation (Eloquent)

```php
<?php
// app/Reservation.php
namespace App;

use Illuminate\Database\Eloquent\Model;

class Reservation extends Model {
    // Champs autorisés à la création en masse
    protected $fillable = ['user_id', 'place_id', 'statut', 'date_debut', 'date_fin'];

    // Relation : une réservation appartient à un utilisateur
    public function user() {
        return $this->belongsTo(User::class);
    }

    // Relation : une réservation est liée à une place
    public function place() {
        return $this->belongsTo(Place::class);
    }
}
```

## 7.2 Contrôleur : soumettre une réservation

```php
// app/Http/Controllers/ReservationController.php

public function store(Request $request) {
    // Validation des données du formulaire
    $request->validate([
        'place_id'   => 'required|exists:places,id',
        'date_debut' => 'required|date|after_or_equal:today',
        'date_fin'   => 'required|date|after:date_debut',
    ]);

    // Création de la réservation avec statut 'en attente'
    Reservation::create([
        'user_id'    => auth()->id(),
        'place_id'   => $request->place_id,
        'statut'     => 'en attente',
        'date_debut' => $request->date_debut,
        'date_fin'   => $request->date_fin,
    ]);

    return redirect()->route('dashboard')
                     ->with('success', 'Votre demande a bien été enregistrée.');
}
```

## 7.3 Middleware IsAdmin

```php
// app/Http/Middleware/IsAdmin.php

public function handle($request, Closure $next) {
    // Vérifie que l'utilisateur connecté a le rôle 'admin'
    if (auth()->check() && auth()->user()->role === 'admin') {
        return $next($request);
```

```
    }
    // Redirection si accès non autorisé
    return redirect('/')->with('error', 'Accès réservé aux administrateurs.');
}
```

## 7.4 Vue Blade — Liste des réservations

```
{{-- resources/views/reservations/index.blade.php --}}
@extends('layouts.app')

@section('content')
<h2>Mes réservations</h2>
@foreach ($reservations as $resa)
    <div class="card">
        <p>Place : {{ $resa->place->numero }}</p>
        <p>Du {{ $resa->date_debut }} au {{ $resa->date_fin }}</p>
        <span class="badge">{{ $resa->statut }}</span>
        @if ($resa->statut === 'en attente')
            <form method="POST" action="/reservation/{{ $resa->id }}">
                @csrf @method('DELETE')
                <button>Annuler</button>
            </form>
        @endif
    </div>
@endforeach
@endsection
```

# 8. Guide d'installation

## 8.1 Prérequis

- PHP >= 7.1 avec les extensions pdo_mysql, mbstring, openssl, tokenizer.
- Composer (gestionnaire de dépendances PHP).
- MySQL ou MariaDB.
- Node.js et npm (optionnel, pour la compilation des assets).

## 8.2 Étapes d'installation

```
# 1. Cloner le dépôt
git clone https://github.com/Rivalzina75/projet_parking.git
cd projet_parking

# 2. Installer les dépendances PHP
composer install

# 3. Copier le fichier de configuration
cp .env.example .env

# 4. Générer la clé d'application
php artisan key:generate

# 5. Configurer la base de données dans .env
# DB_DATABASE=parking
# DB_USERNAME=root
# DB_PASSWORD=

# 6. Exécuter les migrations (création des tables)
php artisan migrate

# 7. (Optionnel) Insérer des données de test
php artisan db:seed

# 8. Lancer le serveur de développement
php artisan serve
# Accès : http://localhost:8000
```

# 9. Tests et recette

## 9.1 Scénarios de test fonctionnel

| Scénario | Actions réalisées | Résultat attendu |
|---|---|---|
| Inscription utilisateur | Remplir le formulaire avec des données valides puis soumettre. | Compte créé, redirection vers le tableau de bord. |
| Connexion invalide | Saisir un mauvais mot de passe. | Message d'erreur, pas de connexion. |
| Demande de réservation | Sélectionner une place disponible et des dates valides. | Réservation créée avec statut 'en attente'. |
| Validation admin | Admin valide une demande en attente. | Statut passe à 'validée', place marquée 'occupée'. |
| Refus admin | Admin refuse une demande. | Statut passe à 'refusée', place reste disponible. |
| Annulation utilisateur | Utilisateur annule une réservation en attente. | Statut passe à 'annulée', place redevient libre. |
| Accès admin sans droit | Un utilisateur standard accède à /admin/dashboard. | Redirection avec message d'erreur (middleware). |

## 9.2 Résultats

**Bilan des tests**

L'ensemble des scénarios de test ont été exécutés manuellement. Les fonctionnalités principales (inscription, connexion, réservation, validation admin, middleware de sécurité) fonctionnent conformément aux spécifications. Le projet est estimé complet à 85 %.

# 10. Bilan et compétences BTS SIO

## 10.1 Compétences du référentiel validées

| Code | Compétence | Mise en œuvre dans le projet |
|---|---|---|
| B1.1 | Gérer le patrimoine informatique | Versioning Git, structure MVC claire |
| B1.3 | Développer la présence en ligne | Application web déployable, interface Blade responsive |
| B1.4 | Travailler en mode projet | Suivi via GitHub, respect du cahier des charges |
| B2.1 | Concevoir et développer une solution applicative | Architecture MVC, contrôleurs, modèles Eloquent |
| B2.2 | Assurer la maintenance | Code documenté, migrations versionnées |
| B2.3 | Gérer les données | BDD MySQL, Eloquent ORM, migrations Laravel |

## 10.2 Ce que j'ai appris

- La mise en pratique du pattern MVC dans un projet réel avec Laravel.
- L'utilisation d'Eloquent ORM pour manipuler la base de données de façon orientée objet.
- La création et l'utilisation de middlewares pour sécuriser les routes.
- La gestion des migrations pour versionner le schéma de base de données.
- L'utilisation des templates Blade pour créer des interfaces dynamiques et sécurisées (protection CSRF).

## 10.3 Difficultés rencontrées

- La prise en main initiale du framework Laravel et de son écosystème (Composer, Artisan).
- La gestion des relations Eloquent et des requêtes imbriquées (eager loading).
- La mise en place du middleware de rôle personnalisé.

## 10.4 Perspectives d'amélioration

- Ajout d'un système de notifications par e-mail lors de la validation ou du refus d'une demande.
- Intégration d'un calendrier interactif pour la sélection des dates de réservation.
- Mise en place de tests automatisés (PHPUnit / Laravel Test).
- Déploiement sur un serveur distant (hébergement web).

# Conclusion

Ce projet de gestion de parking constitue une application web complète développée avec le framework PHP Laravel. Il illustre la maîtrise du pattern MVC, la manipulation d'une base de données relationnelle via un ORM, la sécurisation des accès par authentification et middleware, ainsi que l'utilisation de bonnes pratiques de développement (versioning, migrations, validation).

L'ensemble des fonctionnalités demandées dans le cahier des charges a été implémenté, faisant de ce PPE une réalisation solide pour le portfolio BTS SIO option SLAM.

**Liens du projet**

Dépôt GitHub : https://github.com/Rivalzina75/projet_parking Enoncé PPE : https://enseignement.alexandre-mesle.com/PPE/parking/